Problem Chosen	2022 MCM/ICM Summary Sheet	Team Control Number
C		2216790

Real-time Strategy Decision of Bitcoin and Gold Portfolio: Based on Reinforcement Learning

Summary

Bitcoin and gold are two typical currencies in the world. Bitcoin is a new digital cryptocurrency and gold is a traditional investment currencies. How to balance and get benefit from the market of them is a comprehensive and challenging problem. For this purpose, we built a model based on the reinforcement learning **Proximal Policy Optimization algorithm**.

By using deep reinforcement learning method, we design the state space, action space, and reward function for the portfolio strategy. The input model states include multifaceted and multidimensional information. And we introduce **MACD**, **RSI**, **turbulence** and other technical indicators into our environment. We use a continuous action space instead o.f discrete one. The size of the output values is used to determine the action of the buy or sell as well as the trade size. The reward function is the change in total assets caused by each action step, consistent with our ultimate goal of maximizing total assets.

To make full use of the dataset, we also propose the **train-valid sliding window training method**. The training set uses a growing window with time and the validation set uses a fixed-length window. At the end of a training-trading round, the training set window grows the length of the original validation set and the validation set moves forward. It enables repeated use of the dataset for training and completing five years of trading.

Besides, we find that the prices of gold and bitcoin are correlated, which means that some factors beyond the problem can affect both prices. Hence, there may be an external impact or collapse. To measure this, we introduce turbulence to avoid this situation, which shows the unusual price changes in the market. When it is above a threshold, it means that there is an extreme market condition, and we stop buying and sell all our holdings.

In addition, because the action space is not fixed due to the gold break problem, we add **soft constraint** and **hard constraint** to solve this problem. To avoid the model to converge on points where trades are made during the break, soft constraint amplifies the penalty for a wrong decision. When the model is trading on the validation set, hard constraint forces the model's decision not to change the amount of gold in hand during the break. This approach is a good solution to the nuisance that the gold market break poses to the model training.

For problem 1, we use the well-trained reinforcement learning model. By inputting the current state, the model will give the suggested action to take, which forms our strategy in trade. There is a 45-day wait-and-see period for the model to learn, and then the trading process begins. On the last day, September 10, 2021, our total assets can change from $1,000 to $67,088, which is a quite desirable return. For problem 2, we compare our model results with two baselines in terms of total assets and sharpe ratio. Generally considered, our model has optimal results. For question 3, we see how our fine-tuned model make decisions in a market with various commission rates. By comparing the sharpe ratios and total assets, we find that there is little gap among these situations. Our model have a satisfactory stability.

Keywords: Proximal Policy Optimization; Turbulence; Sliding Windows; Soft Constraint; Hard Constraint

Contents

1 Introduction . . . 3
1.1 Problem Background . . . 3
1.2 Restatement of the Problem . . . 3
1.3 Our Approach . . . 3
2 General Assumptions and Model Overview . . . 4
3 Model Preparation . . . 5
3.1 Notations . . . 5
3.2 The Data . . . 5
3.2.1 Data Filling . . . 6
3.2.2 Moving Average Convergence Divergence . . . 7
3.2.3 Relative Strength Index . . . 7
3.2.4 Trade Availability . . . 7
3.2.5 Turbulence . . . 8
4 Reinforcement Learning . . . 9
4.1 Environments for Gold and Bitcoin . . . 10
4.1.1 State Space . . . 10
4.1.2 Action Space . . . 10
4.1.3 Reward Function . . . 10
4.2 Trading Agent Based on PPO [12] . . . 11
4.3 Details of Model . . . 12
4.4 Performance . . . 12
5 Model Validation . . . 13
5.1 Effectiveness Analysis . . . 13
5.1.1 Compare with Baselines . . . 13
5.1.2 Sharpe Ratio . . . 13
5.2 Sensitivity Analysis . . . 15
6 Conclusions . . . 16
6.1 Summary of Results . . . 16
6.2 Strengths . . . 16
6.3 Possible Improvement . . . 16
7 Memo . . . 17

A Code of Model 20

1 Introduction

1.1 Problem Background

The rise of blockchain technology in the digital economy has led to the emergence and development of a range of crypto-digital currencies like Bitcoin. Traditional and newborn products co-exist in the investment market, two typical cases of which are gold and bitcoin. Since 2011, Bitcoin has experienced many ups and downs of large amount, attracting more trader to pay attention. With the ever-changing prices related to traders' profits, it is critical to balance multiple investment products and develop profitable, risk-resistant investment strategy.

1.2 Restatement of the Problem

- Build a dynamic mathematical model based on data up to that day to invest over a five-year period and ultimately obtains the largest total assets. The model should be dynamically updated on a regular basis and make decisions in real time.
- Given the limited initial capital and the frequent changes in the prices of the two assets over time, it is difficult to find the existent maximum value of returns. Therefore, we should find evidence that our investment strategy is the optimal one. We should identify the metrics used to measure the goodness of the model, including the ability to generate returns and resilience to risk.
- The real trading market is more complex. Lots of factors affecting our decisions, such as changes in fees, correlations between products, etc. Is our model robust and sensitive to cope with these situations? We should explore the responsiveness of the model to different parameter settings.

1.3 Our Approach

We are asked to develop a model to balance the investment between two assets with transaction commission, gold and bitcoin, in order to maximize returns. Based on the data provided, we act as an investment agent to simulate the process of the investment over a five-year period starting from September 11, 2016. Our work mainly includes the following:

- Based on historical prices and trends of gold and bitcoin, we build a reinforcement learning model to determine trading strategy.
- We compare the model's performance with different strategy baselines based on indicators that we propose. And it is obvious that our model have a better strategy.
- We explore the robustness and sensitivity of the model with consideration of more factors and parameters.

2 General Assumptions and Model Overview

To simplify the problem, we make the following basic assumptions, each of them is properly justified.

- **Assumption 1**: Our actions have no contribution to the changes in the gold and bitcoin markets.
  ↪ **Justification**: Though the flow of currencies will have some impact on the market, the influence of many traders with small amount of money like us is negligible
- **Assumption 2**: There is no other factors leading to the change of our assets, other than the cost of transaction commission and the alteration of the price.
  ↪ **Justification**: In this paper, we focus on studying the best strategy in the gold and bitcoin trading process. To simplify the problem, we can use this hypothesis to exclude the impact of other secondary factors.
- **Assumption 3**: In our daily decision making, we follow the principle of selling before buying and only make a maximum of one trade per currency, i.e. only buy, hold or sell once.
  ↪ **Justification**: Only one trade per day for both currencies allows for any feasible combination of holdings and minimizes transaction commissions.

Reinforcement learning is a machine learning method based on feedback from the environment. According to a Markov decision process, a reinforcement learning model receives a state from the environment, outputs an action and subsequently the environment feeds back the reward and the new state to the model.

First, we build an environment that defines a state space, an action space, and a reward function. Then extract the information we need from pre-processed data and transform it into one of the states in the state space. Subsequently, we set up a reinforcement learning model based an actor-critic algorithm: Proximal Policy Optimization (PPO) [12]. The model is trained by feeding the state into the model, getting the action we should take, and rewarding the model's trajectory by the environment.

In Summary, the whole modeling process can be shown as Figure 1.

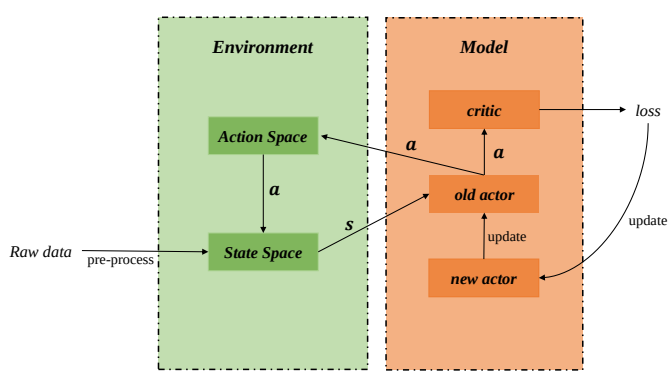

Figure 1: Model Overview

3 Model Preparation

3.1 Notations

Important notations used in this paper are listed in Table 1.

Table 1: Notations

Symbol	Description	Type
C_t	The amount of cash we have at time t	scalar
G_t	The ounces of gold we have at time t	scalar
B_t	The amount of bitcoins we have at time t	scalar
$\boldsymbol{M}_t$	[M_t^{gold},$M_t^{bitcoin}$], MACDs of gold and bitcoin at time t	vector
$\boldsymbol{I}_t$	[I_t^{gold},$I_t^{bitcoin}$], RSIs of gold and bitcoin at time t	vector
$\boldsymbol{Y_t}$	[Y_t^{gold},$Y_t^{bitcoin}$], trade availability of gold and bitcoin at time t	bool
$\boldsymbol{h}_t$	[h_t^{gold},$h_t^{bitcoin}$], hard transaction commissions of gold and bitcoin at time t	vector
$R(\cdot)$	Total reward of a trajectory	function
$\boldsymbol{s}_t$	State at time t	vector
$\boldsymbol{a}_t$	Action at time t	vector
$r(\cdot)$	Reward function, and the reward at time t is denoted as r_t	function
$\pi_\theta(\cdot)$	Policy function fitted by neural network with parameter θ	function
τ	A trajectory generated in a episode, denoted as $< s_1, a_1, s_2, a_2, \ldots, s_T >$	series
$A(\cdot)$	Advantage function, in which the decay coefficient is γ	function
$V(\cdot)$	Value function, to evaluate the future return of state s	function

3.2 The Data

Historical prices for gold and bitcoin from September 11, 2016 to September 11, 2021 are given. Gold trading is closed on weekends and certain holidays, while Bitcoin can be traded every day. Filling NAN at the dates when gold market is closed, we get 1826 days of data for each of gold and bitcoin in total.

To get a reliable, fine-tuned model for trading strategy, we propose a train-valid dataset using a sliding window as Figure 2. The window of train set is growing and the validation set window is fixed-length. The real trade are undertook on the validation set after the model learning on the previous training set. At the end of one validation set, the window moves forward and model starts a new round of train and trade. Ultimately, we finish the trading strategy for five years. This means that at the beginning 45 days we refuse to trade and are in a wait-and-see state.

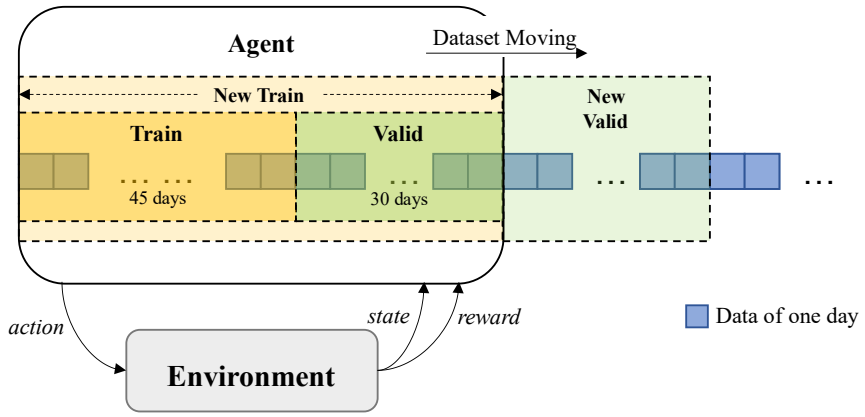

Figure 2: Sliding Window of Train and Valid Process

3.2.1 Data Filling

We find that gold also has some missing price values during market-open periods, which are filled using the average of the ten days before and after. For the days when gold market is closed, the price is filled with the value of the day before the close date. The whole process is shown as Figure 3.

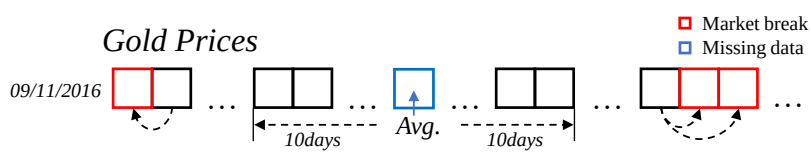

Figure 3: Data Filling Process

3.2.2 Moving Average Convergence Divergence

Moving Average Convergence Divergence (MACD) is used to represent the current trend of the development of stock price. And we calculate it to show the price changing direction of gold and bitcoin. We can get MACD from exponential moving average (EMA) of different periods. For price series $\{p_n^i\}(i = gold, bitcoin)$, EMA of nth item for period N is

$$EMA_N(p_n^i) = \frac{2p_n^i + (N-1)EMA_N(p_{n-1}^i)}{N+1}. \tag{1}$$

And $EMA_N(p_i^i) = p_1^i$. Thus, MACD can be calculated from EMA

$$MACD(p_n^i) = \frac{224}{51}EMA_9(p_n^i) - \frac{16}{3}EMA_{12}(p_n^i) + \frac{16}{17}EMA_{26}(p_n^i). \tag{2}$$

With Equation 2, we compute the MACD values of gold and bitcoin, denote them as $\boldsymbol{M_t}$.

3.2.3 Relative Strength Index

Relative Strength Index (RSI) is used for short-term investment in stocks and can indicate the rise and fall of stocks. In this paper, it indicates the change in the price of gold and bitcoin. It is obtained by calculating relative strength (RS).

Assuming that the daily price rise is U, and drop is D, then at the day when price rises,

$$U = p_t^i - p_{t-1}^i, D = 0.$$

And at the day when price falls,

$$U = 0, D = p_{t-1}^i - p_t^i.$$

So,

$$RS = \frac{EMA_{30}(U)}{EMA_{30}(D)},$$

$$RS = \frac{EMA_{30}(U)}{EMA_{30}(D)}. \tag{3}$$

We use Equation 3 to compute RSI of gold and bitcoin for five years.

3.2.4 Trade Availability

To mark the data of market closing day, we propose the trade availability. When the gold market is closed, the trade availability of gold on that day is set to 0. And the trade availability is the same as the real price at other times. In this way, without changing the original data, the model can get data marked with the information of closed market. Hence, the model has the opportunity to refuse to trade during the gold market closed time.

3.2.5 Turbulence

There are always sudden events in the market that result in large price swings. First, we use Spearman test and Pearson test to prove the prices of gold and bitcoin are relatively correlated. The test results are as follows

Table 2: Spearman Test and Pearson Test

Test	Correlation	p-value
Spearman	0.785	0.0
Pearson	0.649	0.0

It means that the prices could have related trend and mutual influencing factors. For example, they may collapse at the same time. So, we introduce the turbulence factor to control the risk of trading.

$$turbulence_t = (\boldsymbol{y}_t - \boldsymbol{\mu})\boldsymbol{\Sigma}^{-1}(\boldsymbol{y}_t - \boldsymbol{\mu})' \in \mathbb{R}, \tag{4}$$

where $\boldsymbol{y}_t$ denotes the price vector for two currencies at the current moment, $\boldsymbol{\mu}$ denotes the mean of past prices, and $\boldsymbol{\Sigma}$ denotes the covariance matrix of historical prices. When $turbulence_t$ is above a threshold, it means that there is an extreme market condition, and we stop buying and sell all our holdings. [15]

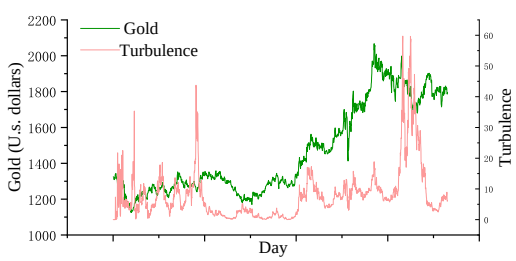

Figure 4: Turbulence and Gold Price

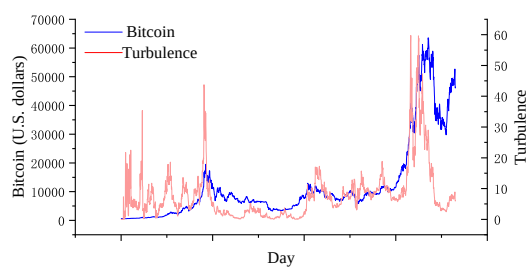

Figure 5: Turbulence and Bitcoin Price

From the turbulence and price figure, we find that when there is a peak in the turbulence series, the corresponding two currency prices are in a period of disorderly and violent fluctuations. Therefore, to reduce the risk, we set the threshold value of the turbulence and when it is exceeded, we adopt a strategy of selling all currencies to hold cash to avoid huge losses.

4 Reinforcement Learning

In recent years, machine learning algorithms have been widely used in many fields, especially in image classification, natural language processing and so on. Similarly, in finance, machine learning algorithms are used to find high-return, low-risk trading strategies. Many articles focus on regression and classification tasks. Here we apply a reinforcement learning algorithm to obtain a comprehensive and effective strategy by inputting information of the gold and bitcoin market state.

To simulate random changes of the dynamic market, we use a Markov reward process $<\boldsymbol{s}, \boldsymbol{a}, r(\cdot), \pi(\cdot), Q(\cdot)>$ to analysis. The value function $V(\boldsymbol{s})$ under the state $\boldsymbol{s}$ can be calculated by Bellman equation

$$V(\boldsymbol{s}) = r(\boldsymbol{s}) + \gamma \sum_{\boldsymbol{s'}} p(\boldsymbol{s'}|\boldsymbol{s}) V(\boldsymbol{s'}), \tag{5}$$

where $p(\boldsymbol{s'}|\boldsymbol{s})$ represents the probability of transition from state $\boldsymbol{s}$ to state $\boldsymbol{s'}$, γ is a decay coefficient. In this paper, the basic process of reinforcement learning algorithm is to update the policy function $\pi_\theta(\cdot)$ with parameter θ through continuous sampling and backpropagation. [14] Assuming the expected reward is

$$\bar{R}_\theta = \mathbb{E}_{\tau \sim p_\theta(\tau)}[R(\tau)], \tag{6}$$

where $R(\tau) = \sum_{t=1}^{T} r_t$ represents the total assets of a decision-making process, $p_\theta(\tau)$ represents the probability distribution of the decision process τ when the parameter is θ.

The objective is to find the policy that maximizes the payoff. Here we take the gradient ascent approach. The gradient of the expected reward is hard to calculate. To simplify, we approximate it by the way of minibatch,

$$\nabla \bar{R}_\theta = \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T_n} R(\tau^n) \nabla log p_\theta(\boldsymbol{a}_t^n | \boldsymbol{s}_t^n). \tag{7}$$

Instead of applying the same weights to the entire decision process for updating. We introduce the advantage function, $A(\boldsymbol{s}_t, \boldsymbol{a}_t) = r(\boldsymbol{s}_t, \boldsymbol{a}_t, \boldsymbol{s}_{t+1}) + \gamma V^\phi(\boldsymbol{s}_{t+1}) - V^\phi(\boldsymbol{s}_t)$, to focus on each action in the decision process. So the gradient becomes

$$\nabla \bar{R}_\theta = \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T_n} A(\boldsymbol{s}_t^n, \boldsymbol{a}_t^n) \nabla log p_\theta(\boldsymbol{a}_t^n | \boldsymbol{s}_t^n). \tag{8}$$

Update the parameters of the policy function neural network by Equation 9.

$$\theta \leftarrow \theta + \eta \nabla \bar{R}_\theta, \tag{9}$$

where η is learning rate, which is a hyperparameter.

4.1 Environments for Gold and Bitcoin

4.1.1 State Space

We use a vector to represent the trader's state at time t: $\boldsymbol{s}_t = [C_t, G_t, B_t, \boldsymbol{p}_t, \boldsymbol{M}_t, \boldsymbol{I}_t, \boldsymbol{Y}_t, \boldsymbol{h}_t]$.

Among them, C_t, G_t, B_t represent the amount of cash, the number of gold ounces, and the number of bitcoins in hand respectively . And the they represent the state of total assets after a transaction.

$\boldsymbol{p}_t$ is a vector representing gold and bitcoin prices, $\boldsymbol{M}_t$ is a vector representing gold and Bitcoin MACDs, $\boldsymbol{I}_t$ is a vector representing gold and bitcoin RSIs of a 30-day window. They describe the changes in gold and bitcoin prices in time series, reflecting price trends and implying turning points.

$\boldsymbol{Y}_t$ represents the trade availability of the gold and bitcoin, which is used to provide the model with a mark of whether the market is closed.

$\boldsymbol{h}_t$ is hard transaction commissions, which represents the commission ratio of an asset when conducting transactions. Unlike $\alpha_{gold}, \alpha_{bitcoin}$, when gold market is closed, $h_t^{gold} = 1$. It makes the model tend to hold gold. If an illegal transaction occurs during the training process, the model wastes a lot of money and will get a lower reward. It's a kind of soft constraints. We make the selling commission of gold become 100% and the buying commission become 500% when the market is closed.

4.1.2 Action Space

Let the action at time t be $\boldsymbol{a}_t \in \mathcal{D}^2$, where $\mathcal{D} = [0, 1]$. The model outputs a vector, whose dimension is 2, representing the action taken at state $\boldsymbol{s}_t$ at time t. The first dimension represents the buying and selling of bitcoin, and the second dimension represents that of gold. Set the hyperparameter λ to represent the maximum amount of each transaction (ounces or number). And we use $\lambda \boldsymbol{a}_t$ to represent the number of gold ounces and bitcoins that should be bought or sold at the current moment.

Instead of using a discrete action space $\{-1, 0, 1\}^2$ to represent actions of sell, hold and buy, a continuous action space can reflect the size of the trading volume. For certain initial capital and multi assets, the continuous action space can better allocate the trade volume and get the optimal portfolio. By limiting the boundary, it is possible to prevent buying and selling in excess of the total assets or holdings.

4.1.3 Reward Function

We define a reward function $r(\cdot)$ to represent the change in total assets leaded by a transition from state $\boldsymbol{s}_t$ to $\boldsymbol{s}_{t+1}$

$$r(\boldsymbol{s}_t, \boldsymbol{a}_t, \boldsymbol{s}_{t+1}) = (C_{t+1} + p_{t+1}^{gold} G_{t+1} + p_{t+1}^{bitcoin} B_{t+1}) - (C_t + p_t^{gold} G_t + p_t^{bitcoin} B_t) - c_t. \quad (10)$$

Among them, c_t represents the transaction commission at time t. If the transaction amount is Δ, then $c_t = \sum p_t^i \alpha_i |\Delta_i| (i = gold, bitcoin)$.

This is consistent with our ultimate goal. In order to maximize the total assets, the change of it needs to be set as the reward function of the model.

4.2 Trading Agent Based on PPO [12]

Proximal Policy Optimization (PPO) [12] is an actor-critic structure reinforcement learning algorithm with three networks. They are new-actor, old-actor, and critic, where the actor is responsible for interacting with the environment, and the critic is responsible for scoring the behavior of the actor.

Note that new-actor, old-actor and critic are respectively $\pi_\theta(\cdot), \pi_{\theta_{old}}(\cdot), V^\phi(\cdot)$. Then in PPO [12], we want to sample from the old-actor and use it to update the new-actor network, so the gradient of the expected reward becomes

$$\nabla \bar{R}_\theta = \mathbb{E}_{(\boldsymbol{s}_t,\boldsymbol{a}_t)\sim\pi_{\theta_{old}}}\left[\frac{p_\theta(\boldsymbol{a}_t|\boldsymbol{s}_t)}{p_{\theta_{old}}(\boldsymbol{a}_t|\boldsymbol{s}_t)}\hat{A}(\boldsymbol{s}_t,\boldsymbol{a}_t)\nabla log p_\theta(\tau)\right], \tag{11}$$

where $\frac{p_\theta(\boldsymbol{a}_t|\boldsymbol{s}_t)}{p_{\theta_{old}}(\boldsymbol{a}_t|\boldsymbol{s}_t)}$ is the weight in important sampling, and the advantage function $\hat{A}(\cdot)$ is simulated by critic network. Therefore the clipped surrogate objective function of PPO [12] is

$$J^{CLIP}(\theta) = \sum_{(\boldsymbol{s}_t,\boldsymbol{a}_t)} min\left(\frac{p_\theta(\boldsymbol{a}_t|\boldsymbol{s}_t)}{p_{\theta_{old}}(\boldsymbol{a}_t|\boldsymbol{s}_t)}\hat{A}(\boldsymbol{s}_t,\boldsymbol{a}_t), clip\left(\frac{p_\theta(\boldsymbol{a}_t|\boldsymbol{s}_t)}{p_{\theta_{old}}(\boldsymbol{a}_t|\boldsymbol{s}_t)}, 1-\epsilon, 1+\epsilon\right)\hat{A}(\boldsymbol{s}_t,\boldsymbol{a}_t)\right).$$

The function of the clip term is to limit the distance between the old-actor and the new-actor distribution.

The algorithm flow is as follows

Algorithm 1 PPO [12]

```
repeat
    Initialize parameters θ, ϕ for actor network π_θ(·) and critic network V_ϕ(·)
    repeat
        The old actor network π_θ_old(·) interact with environment
        Estimate advantage Â_i (i = 1, 2, ··· , T)
    until Reach the max timesteps T
    Optimize surrogate L wrt θ and ϕ
    Update θ_old to θ
until Reach the max iteration number
```

4.3 Details of Model

For the case of gold and bitcoin in the problem, we need to add some details to get better results.

- **Market Break**
  First of all, we need to mark the time when the market is closed. In order to ensure the normal operation of the model, we will input the gold price at the close time instead of NAN. Meanwhile, the trade availability dimension is added in the state to mark whether the market is closed or not. If gold market is in a closed state, then the trade availability of gold at this moment is the outlier 0.

  In the training process, we use soft constraints to prevent the model from trading during the closed period of gold. Concretely, we give a significant penalty to buy and sell actions to optimize the model in the direction away from that decision. During trading, hard constraints are used to cancel all operations performed on gold when the market is closed. Thus, we solve the problem of uncertainty of action space.

- **Trading Amount**
  We set the parameter λ to control the upper limit of the amount of a single trade to avoid consuming too much assets in commissions due to excessive trade size, reducing the risk of the model.

4.4 Performance

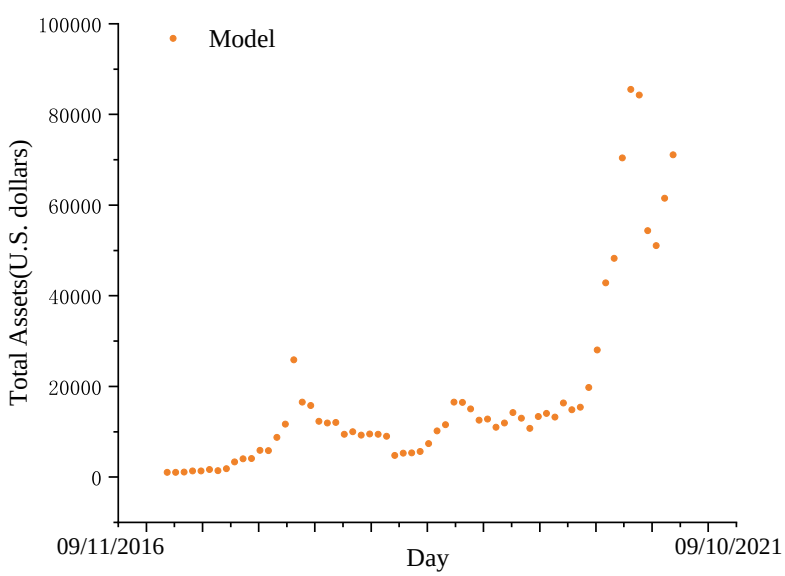

Figure 6: Performance

We implement our model of reinforcement learning by using a python third-party library, *stable baselines* [6]. And our model's performance is as Figure 6. It can be noticed that the model can reach a maximum total assets of about $100,000 and a total asset of $67,088 at the last moment. With initial capital of $1000, the profit is enormous. It shows that our model still has a good return with the consideration of the risk. The calculation of total assets is

$$Total\ assets_t = C_t + B_t p_t^{bitcoin} + G_t p_t^{gold}. \tag{12}$$

The trend of total assets is corresponding to the price trend of two currencies. And the trend of change receives more influence from Bitcoin. For example, in the later stages, total assets and Bitcoin price show a similar trend of rapid increase. This proves that our model's decision will be more focused on high priced and fast changing currencies. This is also consistent with the actual decisions.

5 Model Validation

5.1 Effectiveness Analysis

5.1.1 Compare with Baselines

We compare our model with the following baselines, including some commonly used quantitative trading strategies, comparison is in Table 3.

- **Buy and Hold**

  Buy and hold means we buy gold and bitcoin at the beginning and hold it in our hands until the end of the five-year period. The payoff for this strategy is the price change of the market. Obviously, no matter what the ratio of gold and bitcoin bought is, the final total assets must be in the interval [$1354.8, $74589.7]. Total assets of paying all cash on gold is $1354.8, and that of bitcoin is $74589.7.

- **Chase up and down**

  Buy when the price of the day is higher than the highest price in the past ten days, and sell when the price of the day is lower than the lowest price in the past ten days. The volume of each transaction is 40% of cash.

  In Table 3, we compare the total assets and average sharpe ratio of these three strategies.

5.1.2 Sharpe Ratio

The Sharpe ratio reflects the return on investment compared to risk. The higher the Sharpe ratio, the higher the return per unit risk of the trade.

$$sharpe\ ratio = \frac{\mathbb{E}(R)}{\sigma(R)},$$

where $\mathbb{E}(R)$ is the expectation of historical returns and $\sigma(R)$ is the standard deviation of returns.

Our model results and baselines results are as Table 3, where our baselines focus on bitcoin that brings more profit. It is obvious that our average sharpe ratio is the largest, which indicates that we have the biggest return per unit of risk. The strategy Buy and Hold have a similar result to our model. That may suggest that our model learn the strategy of Buy and Hold. However, due to many factors we take into accounts, it is believed that our model can handle more complex situation.

Table 3: Strategy Comparison

Strategy	Total Assets	Average Sharpe Ratio
PPO	\$67088.42	0.10
Buy&Hold (Bitcoin)	\$74589.69	0.09
Chase up&down	\$11815.24	0.0005

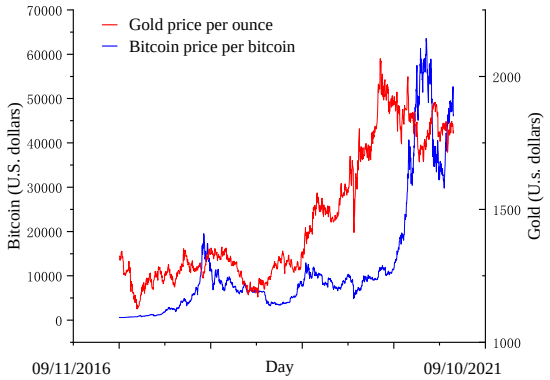

Figure 7: Prices of Bitcoin and Gold

The final state of cash, bitcoin and gold ounces is [0, 1.456, 0]. Given Bitcoin's huge gains during the final forecast phase, we believe that holding only Bitcoin is a reasonable return decision.

5.2 Sensitivity Analysis

We use different transaction commissions to test the model's sensitivity. Analyzing the model from total assets and sharpe ratio. As Table 4 and Figure 8, we can notice that the changes of both total assets and sharpe ratio are within an acceptable range.

Table 4: Total Assets at Different Commissions

$\alpha_{bitcoin}$	α_{gold}	Total Assets
0.01	0.01	\$67738.84
0.01	0.02	\$67727.73
0.02	0.01	\$67088.42
0.02	0.02	\$67065.38
0.04	0.02	\$65825.12
0.2	0.1	\$57234.69

- **Stability**
  Different fee settings all end up with higher returns, demonstrating the stability of the model for different trading environments with optimal decisions.

- **Sensitivity**
  Different parameter settings result in different means and variances of the Sharpe ratio, indicating that the parameter settings cause the model decisions to change accordingly. For example, when we set higher rates in equal proportions, the variance of the Sharpe ratio increases and the return decreases, which is also consistent with the reality.

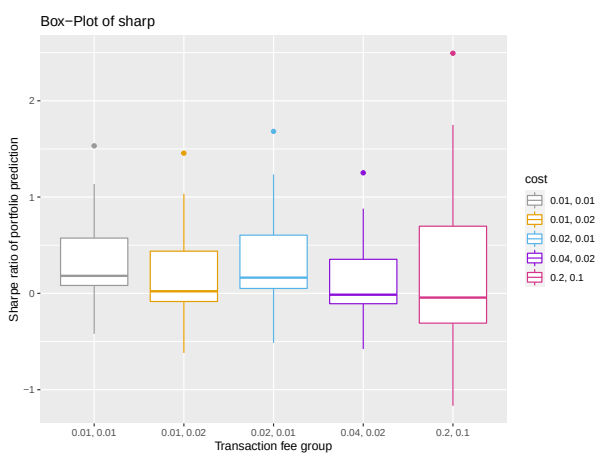

Figure 8: Boxes of Sharpe Ratio at various commissions

6 Conclusions

6.1 Summary of Results

For problem 1, our model invests $1000 from September 2016, and the rate of return changes over time as Figure 6. After five years of investment, our final total assets reached $67088.42.

For problem 2, compared with baselines as Table 3, analyzing the sharpe ratio and total return, we evaluate the effectiveness of the model, and prove that our model is an excellent model that comprehensively considers many aspects. With the model, we can give a more effective and stable strategy.

For problem 3, using different transaction commissions, we find that both total assets and sharpe ratio have reasonable difference. It shows that the transaction commissions have little impact on the overall effect of model strategy. Our model can provide a stable and robust strategy for traders.

By analyzing the results, we find that this model has a higher propensity to trade for higher priced currencies. Also due to the turbulence factor setting, our model has the safety in case of a major market shock.

6.2 Strengths

- We use machine learning algorithms that have rapidly developed in recent years. It helps us in decision-making and analysis through reinforcement learning models.
- We make full use of the data by setting the sliding window of the dataset. Besides, we use the soft constraints method during the training process to prevent the model from trading during the market break, and use the hard constraints method during the trading process to force the model to stop trading during the market break.
- Since the reinforcement learning model is based on experience and simulation of future states and actions, this model does not need to predict future prices, but rather makes decisions in real time with daily data, which is highly time-sensitive.

6.3 Possible Improvement

- Through a rough estimation, the max profit can reach about $10,000,000 or more. However, our model can only reach $67,088.42. Increasing the total assets and sharpe ratio to make the model's return per unit risk become larger is the point worth improving.
- The training speed of the PPO algorithm [12] model is relatively slow. It takes a long time to make a episode of decisions, which lags the pace of our analysis and improvement.

7 Memo

Dear, Mr. trader

In the field of quantitative trading, there are many different traditional strategies like Bollinger Bands, SMAs, Momentum, etc. In recent years, the rise of machine learning algorithms has also activated the field of quantitative trading to experiment with new models. We desire to use a brand-new strategy, reinforcement learning, to solve the problem of trading in the bitcoin and gold markets.

Our model is built on the Proximal Policy Gradient algorithm [12], which uses a sliding window for putting the training and trading data forward in time. From September 11, 2016, the model's strategy earned from $1,000 to $67088.42 in total assets over a five-year period. Considering and analyzing the modeling process and model outputs together, we summarize several details of strategy for you.

First, consider a variety of short-term indicators, such as MACD, RSI, etc. Short-term indicators can reflect the price direction and rise or fall in a shorter period of time. It can also effectively give the lastest condition of the asset. If you use long-term indicators, you will take into account the far-away historical data that have become invalid. And there is a possibility that you will make wrong decision by accepting too much useless information.

Secondly, some indicators that react to particular changes in prices are used to observe the occurrence of anomalies. We set the turbulence indicator to help the model exit in time when market conditions are abnormal to avoid further losses. Turbulence indicator responds well to abnormal price movements. When it is above a threshold, our model sells all its holdings and stops buying operations, entering a wait-and-see state. And we retrade when the price becomes stable or some profitable opportunity emerges.

Thirdly, with such considerations above, you can try the buy and hold strategy. It is similar to our models strategy, for both total assets and sharpe ratio are very close. And the bitcoin market is promising as our model prefer to buy bitcoins.

Also, you can get rid of the influence of transaction commissions when choosing strategy, as long as it is within a reasonable range. We have tried many pairs of commissions and finally find that there is no significant effect on our strategy. Our model will give you a profitable, low-risk and stable trading strategy.

Last but not least, remember to control the amount of each transaction. We set a hyperparameter to limit the model's range of trade amount. It prevents the model from falling into extreme values. And for traders, it ensures the safety of capital.

MCM Team # 2216790.

References

[1] Time series momentum. 2022.

[2] Wei Bao, Jun Yue, and Yulei Rao. A deep learning framework for financial time series using stacked autoencoders and long-short term memory. *PloS one*, 12(7):e0180944, 2017.

[3] Jamil Baz, Nicolas M. Granger, Campbell R. Harvey, Nicolas Le Roux, and Sandy Rattray. Dissecting investment strategies in the cross section and time series. *Social Science Research Network*, 2015.

[4] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

[5] Prafulla Dhariwal, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, Yuhuai Wu, and Peter Zhokhov. Openai baselines (2017). *URL https://github. com/openai/baselines*, 2016.

[6] Ashley Hill, Antonin Raffin, Maximilian Ernestus, Adam Gleave, Anssi Kanervisto, Rene Traore, Prafulla Dhariwal, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, and Yuhuai Wu. Stable baselines. `https://github.com/hill-a/stable-baselines`, 2018.

[7] Mark Kritzman and Yuanzhen Li. Skulls, financial turbulence, and risk management. *Financial Analysts Journal*, 66(5):30–41, 2010.

[8] Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance. *Deep RL Workshop, NeurIPS 2020*, 2020.

[9] Cong Ma, Jiangshe Zhang, Junmin Liu, Lizhen Ji, and Fei Gao. A parallel multi-module deep reinforcement learning algorithm for stock trading. *Neurocomputing*, 449:290–302, 2021.

[10] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

[11] Emad W Saad, Danil V Prokhorov, and Donald C Wunsch. Comparative study of stock trend prediction using time delay, recurrent and probabilistic neural networks. *IEEE Transactions on neural networks*, 9(6):1456–1470, 1998.

[12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[13] Weiyu Si, Jinke Li, Peng Ding, and Ruonan Rao. A multi-objective deep reinforcement learning approach for stock index future's intraday trading. In *2017 10th International symposium on computational intelligence and design (ISCID)*, volume 2, pages 431–436. IEEE, 2017.

[14] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999.

[15] Hongyang Yang, Xiao-Yang Liu, Shan Zhong, and Anwar Walid. Deep reinforcement learning for automated stock trading: An ensemble strategy. In *Proceedings of the First ACM International Conference on AI in Finance*, pages 1–8, 2020.

[16] Yong Zhang and Xingyu Yang. Online portfolio selection strategy based on combining experts' advice. *Computational Economics*, 50(1):141–159, 2017.

[17] Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deep reinforcement learning for trading. *The Journal of Financial Data Science*, 2(2):25–40, 2020.

A Code of Model

```
class StockEnvTrade(gym.Env):
"""A stock trading environment for OpenAI gym"""
metadata = {'render.modes': ['human']}

def __init__(self, df,day = 0,initial=True, previous_state=[], model_name='', iteratio
    #super(StockEnv, self).__init__()
    #money = 10 , scope = 1
    self.day = day
    self.df = df

    self.initial = initial
    self.previous_state = previous_state
    # action_space normalization and shape is STOCK_DIM
    self.action_space = spaces.Box(low = -1, high = 1,shape = (STOCK_DIM,))

    self.observation_space = spaces.Box(low=0, high=np.inf, shape = (13,))
    # load data from a pandas dataframe
    self.data = self.df.loc[self.day,:]
    self.terminal = False
    # initalize state
    self.state = [INITIAL_ACCOUNT_BALANCE] + \
                  self.data.price.values.tolist() + \
                  [0]*STOCK_DIM + \
                  self.data.real.values.tolist()+\
                  self.data.hard_trans.values.tolist()+\
                  self.data.macd.values.tolist() + \
                  self.data.rsi30.values.tolist()
    # initialize reward
    self.reward = 0
    self.turbulence_threshold=46
    self.turbulence=0
    self.cost = 0
    self.trades =[0,0]
    # memorize all the total balance change
    self.asset_memory = [INITIAL_ACCOUNT_BALANCE]
    self.rewards_memory = []
    self.action_memory=[]
    #self.reset()
    self._seed()
    self.model_name=model_name
    self.iteration=iteration


def _sell_stock(self, index, action):
    # perform sell action based on the sign of the action

        if self.state[index+1]==0:
            pass
        else:
            if self.turbulence<self.turbulence_threshold:
                if self.state[index+STOCK_DIM+1] > 0:
                #update balance
```

```
                    self.state[0] += \
                    self.state[index+5]*min(abs(action),self.state[index+STOCK_DIM+1])
                        (1- self.state[index+7])

                    self.state[index+STOCK_DIM+1] -= min(abs(action), self.state[index-
                    self.cost += self.state[index+5] *min(abs(action),self.state[index-
                            self.state[index+7]
                    self.trades[index]+=1
                else:
                    pass
            else:
                if self.state[index+STOCK_DIM+1] > 0:
                    self.state[0] += self.state[index+5]*self.state[index+STOCK_DIM+1]*
                              (1- self.state[index+7])
                    self.state[index+STOCK_DIM+1] =0
                    self.cost += self.state[index+1]*self.state[index+STOCK_DIM+1]* \
                          self.state[index+7]
                    self.trades[index]+=1
                else:
                    pass

    def _buy_stock(self, index, action):
        # perform buy action based on the sign of the action

            if self.state[index+1]==0:
                pass
            else:
                if self.turbulence>=self.turbulence_threshold:
                    pass
                else:
                    available_amount = self.state[0] / (self.state[index+5]*(1+self.state[i
            # print('available_amount:{}'.format(available_amount))

            #update balance
                    self.state[0] -= self.state[index+5]*min(available_amount, action)* \
                            (1+ self.state[index+7])

                    self.state[index+STOCK_DIM+1] += min(available_amount, action)

                    self.cost+=self.state[index+5]*min(available_amount, action)* \
                            self.state[index+7]
                    self.trades[index]+=1


    def step(self, actions):

        self.terminal = self.day >= len(self.df.index.unique())-1

        if self.terminal:
            end_total_asset = self.state[0]+ \
            sum(np.array(self.state[5:(STOCK_DIM+5)])*np.array(self.state[(STOCK_DIM+1):(ST
            print("previous_total_asset:{}".format(self.asset_memory[0]))

            df_total_value.columns = ['account_value']
```

```
        df_total_value['daily_return']=df_total_value.pct_change(1)

        df_rewards = pd.DataFrame(self.rewards_memory)
        df_rewards.to_csv('results/'+s+'/rewards/account_rewards_trade_{}_{}.csv'.forma

        action_memory=pd.DataFrame(self.action_memory)
        action_memory.to_csv('results/'+s+'/action/account_rewards_trade_{}_{}.csv'.for
        print('total asset: {}'.format(self.state[0]+ sum(np.array(self.state[1:29])*np

        return self.state, self.reward, self.terminal,{}

    else:

        actions = actions * HMAX_NORMALIZE

       # judge whether trade is allowed
        for i in range(2):
            if self.state[i+1]==0:
                actions[i]=0

        self.action_memory.append(actions)
        begin_total_asset = self.state[0]+ \
        sum(np.array(self.state[5:(STOCK_DIM+5)])*np.array(self.state[(STOCK_DIM+1):(ST
        #print("begin_total_asset:{}".format(begin_total_asset))

        argsort_actions = np.argsort(actions)

        sell_index = argsort_actions[:np.where(actions < 0)[0].shape[0]]
        buy_index = argsort_actions[::-1][:np.where(actions > 0)[0].shape[0]]

        for index in sell_index:
            self._sell_stock(index, actions[index])

        for index in buy_index:
            self._buy_stock(index, actions[index])

        self.day += 1
        self.data = self.df.loc[self.day,:]
        self.turbulence=self.data['turbulence'].values[0]

        self.state =  [self.state[0]] + \
                 self.data.price.values.tolist() + \
                list(self.state[(STOCK_DIM+1):(STOCK_DIM*2+1)]) + \
                self.data.real.values.tolist()+\
                self.data.hard_trans.values.tolist()+\
                self.data.macd.values.tolist() + \
                self.data.rsi30.values.tolist()


        end_total_asset = self.state[0]+ \
        sum(np.array(self.state[5:(STOCK_DIM+5)])*np.array(self.state[(STOCK_DIM+1):(ST
        self.asset_memory.append(end_total_asset)

        self.reward = end_total_asset - begin_total_asset
```

```
            self.rewards_memory.append(self.reward)

            self.reward = self.reward*REWARD_SCALING


        return self.state, self.reward, self.terminal, {}

    def reset(self):
        if self.initial:
            self.asset_memory = [INITIAL_ACCOUNT_BALANCE]
            self.day = 0
            self.data = self.df.loc[self.day,:]
            self.turbulence = 0
            self.cost = 0
            self.turbulence=0
            self.trades = [0,0]
            self.terminal = False
            #self.iteration=self.iteration
            self.rewards_memory = []
            #initiate state
            self.state = [INITIAL_ACCOUNT_BALANCE] + \
                         self.data.price.values.tolist() + \
                      [0]*STOCK_DIM + \
                      self.data.real.values.tolist()+\
                      self.data.hard_trans.values.tolist()+\
                      self.data.macd.values.tolist() + \
                      self.data.rsi30.values.tolist()
        else:
            previous_total_asset = self.previous_state[0]+ \
            sum(np.array(self.previous_state[5:(STOCK_DIM+5)])*np.array(self.previous_state
            self.asset_memory = [previous_total_asset]
            #self.asset_memory = [self.previous_state[0]]
            self.day = 0
            self.data = self.df.loc[self.day,:]

            self.cost = 0
            self.trades = [0, 0]
            self.terminal = False

            self.rewards_memory = []

            self.state = [ self.previous_state[0]] + \
                          self.data.price.values.tolist() + \
                          self.previous_state[(STOCK_DIM+1):(STOCK_DIM*2+1)]+ \
                          self.data.real.values.tolist()+\
                          self.data.hard_trans.values.tolist()+\
                          self.data.macd.values.tolist() + \
                          self.data.rsi30.values.tolist()
            return self.state
```